

Project Documentation

Bug Reporter fullstack app

Students:

Staver Maxim, Știr Cezar-Alexandru, Tulbure Robert-Alexandru

University: Technical University of Cluj-Napoca

Course: Software Design / Proiectare Software (PS)

Academic Year: 2025–2026, Semester 2

1 Project Title and Abstract

Abstract

The **Bug Reporter** system is a full-stack web application built to help development teams track and resolve software bugs more effectively. Often, teams struggle with disorganised communication when managing defects, making it hard to prioritise issues or keep track of who is responsible for fixing them. This project solves that problem by offering a dedicated platform where users can easily submit structured bug reports. These reports can include detailed descriptions, image attachments, tags, and status updates. The system allows team members to comment on and vote on submitted bugs.

The system is implemented as a **distributed architecture** comprising three independently deployable components: a **React TypeScript single-page application** serving as the user-facing frontend, a **Java Spring Boot monolith** that encapsulates the core business logic (user management, bug and comment CRUD, voting and reputation scoring, authentication, and moderation), and a dedicated **Spring Boot notification microservice** responsible for dispatching email and SMS notifications upon moderation events. The monolith communicates with the microservice via synchronous **REST API** calls made asynchronously through Spring's **@Async** mechanism, thereby decoupling the moderation workflow from the notification delivery pipeline.

Beyond the baseline requirements, the system implements two bonus feature sets: a **reputation scoring engine** that computes cumulative user scores based on community upvotes and downvotes across bugs and comments, and a **moderation subsystem** that empowers privileged moderator-role users to ban or unban accounts, edit or remove any content, and trigger multi-channel (email and SMS) notifications to banned users via the notification microservice. The frontend enforces role-based access control through protected routes and an Axios interceptor layer, while the backend secures all endpoints with **Spring Security** session-based authentication and a custom **BannedUserFilter** that intercepts requests from banned accounts mid-session.

2 System Architecture and Technology Stack

2.1 High-Level Architecture

The Bug Reporter system follows a **client-server distributed architecture** composed of three independently running services:

1. **Frontend (React SPA)** — A single-page application built with React 19 and TypeScript, bundled by Vite 8. It runs on <http://localhost:5173> during development and communicates exclusively with the monolith backend via RESTful HTTP requests using **Axios** with credential-bearing cookies for session management.
2. **Backend Monolith (Spring Boot)** — A Java 21 Spring Boot 4.0.4 application running on <http://localhost:8080>. It serves as the system's authoritative data layer and business logic engine, exposing REST endpoints for authentication, user management, bug and comment CRUD, voting, filtering, and moderation. It persists data in a **PostgreSQL** relational database (`bug_reporter_db`) via **Spring Data JPA** and **Hibernate ORM**. The monolith also acts as a REST client when it needs to dispatch notifications to the microservice.
3. **Notification Microservice (Spring Boot)** — A lightweight, single-purpose Java 21 Spring Boot 4.0.4 application running on <http://localhost:8081>. Its sole domain responsibility is the delivery of notification messages through two channels: **email** (via

Spring Mail connected to a local MailHog SMTP server on port 1025) and **SMS** (via HTTP POST to a mock SMSPitt server on port 2876). It maintains its own **PostgreSQL** database (`notification_db`) to persist an audit trail of all notification delivery attempts and their statuses (SENT or FAILED).

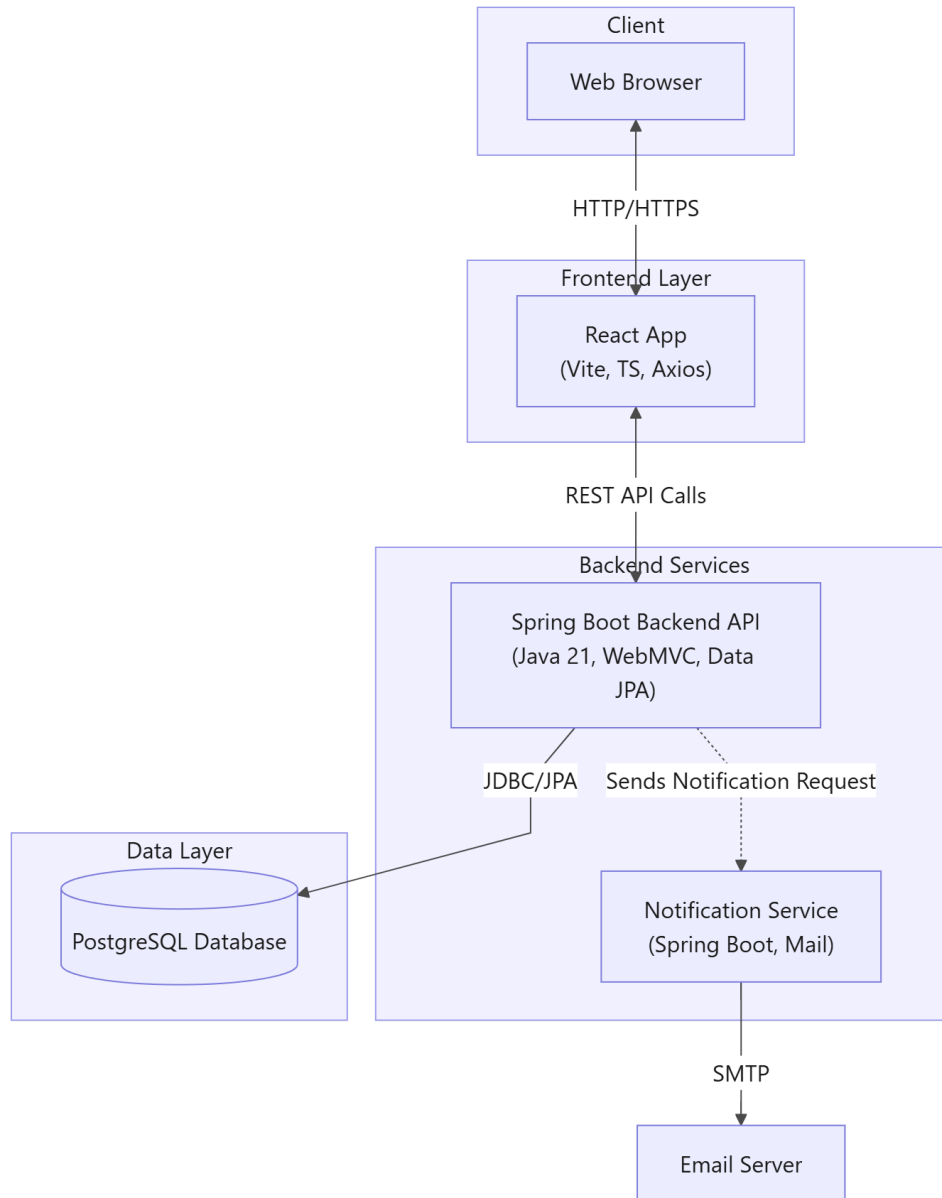


Figure 1: System Architecture Diagram

Data Flow

The typical request lifecycle proceeds as follows:

1. The **React frontend** issues an HTTP request (e.g., `POST /moderation/users/{id}/ban`) to the monolith at <http://localhost:8080>, including the `JSESSIONID` session cookie for authentication.
2. The **Spring Security filter chain** on the monolith authenticates the session, verifies

that the user holds the **MODERATOR** role, and checks via the **BannedUserFilter** that the requesting user is not themselves banned.

3. The **ModerationController** delegates to the **ModerationService**, which updates the target user's **isBanned** flag in the PostgreSQL database.
4. The **ModerationService** invokes the **NotificationClient**, which — annotated with **@Async** — fires a non-blocking **RestTemplate** POST request to the notification microservice at <http://localhost:8081/notifications>.
5. The **Notification Microservice** receives the request, dispatches the email via **JavaMailSender** to MailHog and the SMS via **RestTemplate** to SMSpitt, persists the notification record in its own database, and returns a **201 Created** response.
6. The monolith returns a success response to the frontend, which updates the UI accordingly.

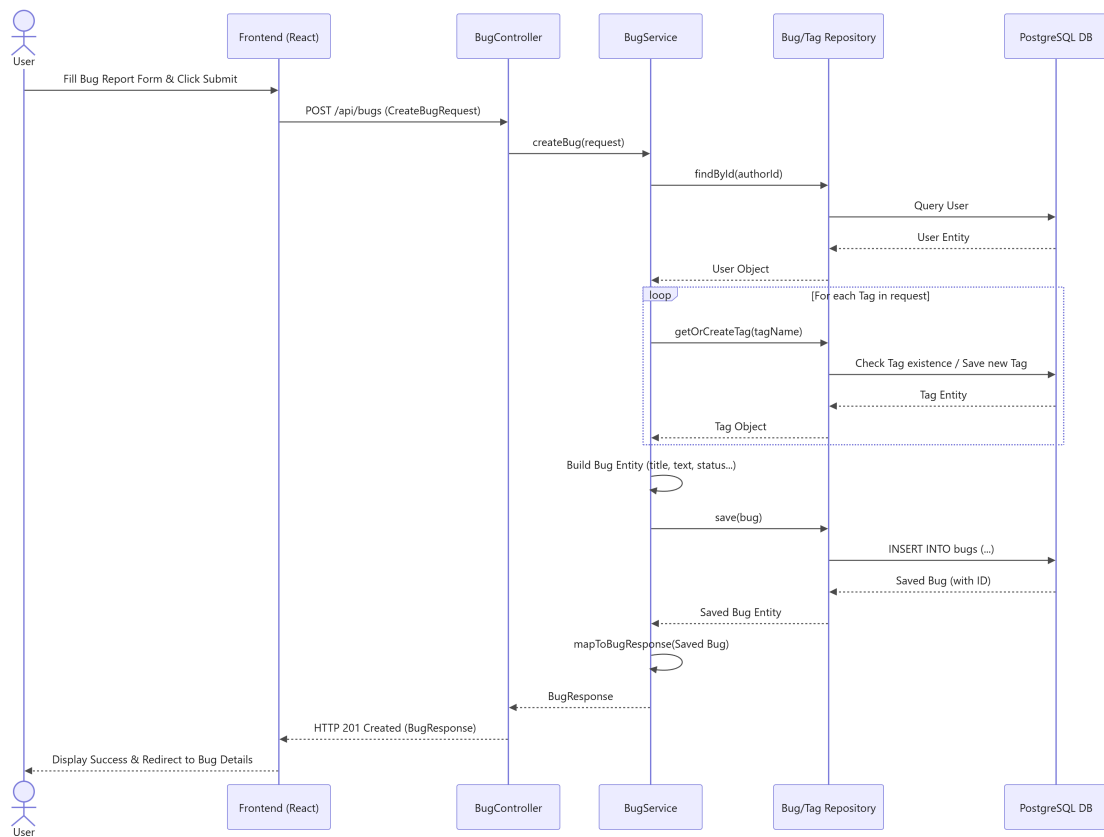


Figure 2: Sequence Diagram for reporting a bug

2.2 Technology Stack

Backend Monolith

Technology	Version	Purpose
Java	21 (LTS)	Primary programming language
Spring Boot	4.0.4	Application framework and dependency injection
Spring Web MVC	<i>starter</i>	REST API controller layer
Spring Data JPA	<i>starter</i>	ORM and repository abstraction
Spring Security	<i>starter</i>	Session-based authentication and authorisation
Spring Validation	4.1.0-M3	Bean validation with Jakarta annotations
Hibernate	<i>managed</i>	JPA implementation and DDL auto-generation
PostgreSQL Driver	<i>managed</i>	JDBC connectivity to PostgreSQL
Lombok	<i>managed</i>	Boilerplate reduction (@Data, @Builder, etc.)
SpringDoc OpenAPI	3.0.3	Swagger UI and OpenAPI 3.0 specification generation
BCrypt	<i>Security</i>	Password hashing
Maven	<i>wrapper</i>	Build automation and dependency management

Notification Microservice

Technology	Version	Purpose
Java	21 (LTS)	Primary programming language
Spring Boot	4.0.4	Application framework
Spring Web	<i>starter</i>	REST API endpoint
Spring Data JPA	<i>starter</i>	Notification persistence
Spring Mail	<i>starter</i>	SMTP email dispatch
Spring Validation	<i>starter</i>	Request payload validation
PostgreSQL Driver	<i>managed</i>	JDBC connectivity
Lombok	<i>managed</i>	Boilerplate reduction
MailHog	External	Local SMTP server and email testing UI
SMSPPitt	npm pkg	Mock SMS gateway

Frontend

Technology	Version	Purpose
React	19.2.4	UI component library
TypeScript	6.0.2	Static type safety
Vite	8.0.4	Development server and build tool
React Router DOM	7.14.0	Client-side routing
Axios	1.15.0	HTTP client with interceptors
Lucide React	1.8.0	Icon library
React Markdown	10.1.0	Markdown rendering in bug/comment text
React Syntax High.	16.1.1	Code block syntax highlighting
Cypress	15.15.0	End-to-end testing framework
ESLint	9.39.4	Code linting

Infrastructure and Databases

2.3 Prerequisites

Ensure the following software is installed and available on the system **PATH**:

Technology	Purpose
PostgreSQL	Relational database engine (two databases: bug_reporter_db, notification_db)
MailHog	Local SMTP mail capture server (port 1025 SMTP, port 8025 Web UI)
SMSPitt	Mock SMS delivery server (port 2875 Web UI, port 2876 API)

Prerequisite	Minimum Version	Verification Command
Java JDK	21	java -version
Apache Maven	3.9+	mvn -version
Node.js	18+ (LTS)	node -v
npm	9+	npm -v
PostgreSQL	14+	psql --version
MailHog	Latest	Downloaded binary or Docker image
SMSPitt	Latest (npm)	npx smpitt --help

3 Backend Implementation

3.1 System Design

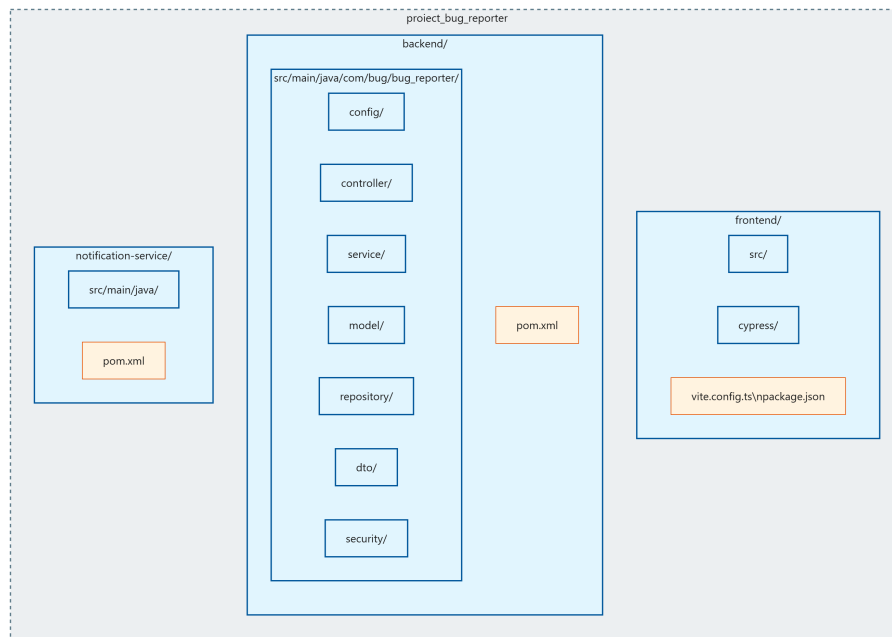


Figure 3: Backend Package Diagram

The backend monolith adheres to a **layered architecture** that implements several well-established design patterns:

Model–View–Controller (MVC)

The application strictly separates concerns into three layers:

- **Controller Layer** (controller/) — Eight `@RestController` classes that define the REST API surface. Each controller handles HTTP request mapping, delegates to the service layer, and returns `ResponseEntity` objects.

- **Service Layer** (`service/`) — Six `@Service` classes encapsulating all business logic, including entity creation, validation, score computation, vote toggling, access control checks, and inter-service communication.
- **Model / Entity Layer** (`model/`) — Seven JPA `@Entity` classes (`User`, `Bug`, `Comment`, `Tag`, `BugVote`, `CommentVote`, `UserRole`) that define the relational schema.

Repository Pattern

The data access layer is abstracted through **Spring Data JPA repositories** (`repository/`). Six interfaces extend `JpaRepository`, providing standard CRUD operations plus custom query methods.

Data Transfer Object (DTO) Pattern

The system defines 15 DTO classes (`dto/`) to decouple the internal entity model from the API contract. Inbound DTOs (e.g., `CreateBugRequest`) carry validated client payloads, while outbound DTOs (e.g., `BugResponse`) shape the JSON responses.

Global Exception Handling

A `@RestControllerAdvice` class (`GlobalExceptionHandler`) centralises error handling across all controllers.

Security Architecture

The security layer employs **session-based authentication** via Spring Security:

- `SecurityConfig` — Configures the `SecurityFilterChain` with CSRF disabled, CORS whitelisting, and endpoint-level authorisation rules.
- `CustomUserDetailsService` — Loads users from the database by username.
- `CustomUserDetails` — Wraps the `User` entity to expose `GrantedAuthority` objects and the `isAccountNonLocked()` flag.
- `BannedUserFilter` — A `OncePerRequestFilter` that checks if a user has been banned since their session was created.
- `@PreAuthorize` / `SecurityUtils` — Method-level security enforces resource ownership or moderator rights.

3.2 Microservice Architecture

Domain Responsibility

The **Notification Microservice** has a single, well-defined domain responsibility: **delivering multi-channel notifications** (email and SMS) and maintaining an audit log.

Communication Protocol

The monolith communicates with the microservice via **synchronous REST over HTTP**, implemented using Spring's `RestTemplate`:

- **Client Side (Monolith):** The `NotificationClient` issues a POST request to <http://localhost:8081/notifications>. Annotated with `@Async`, it executes on a separate thread pool.

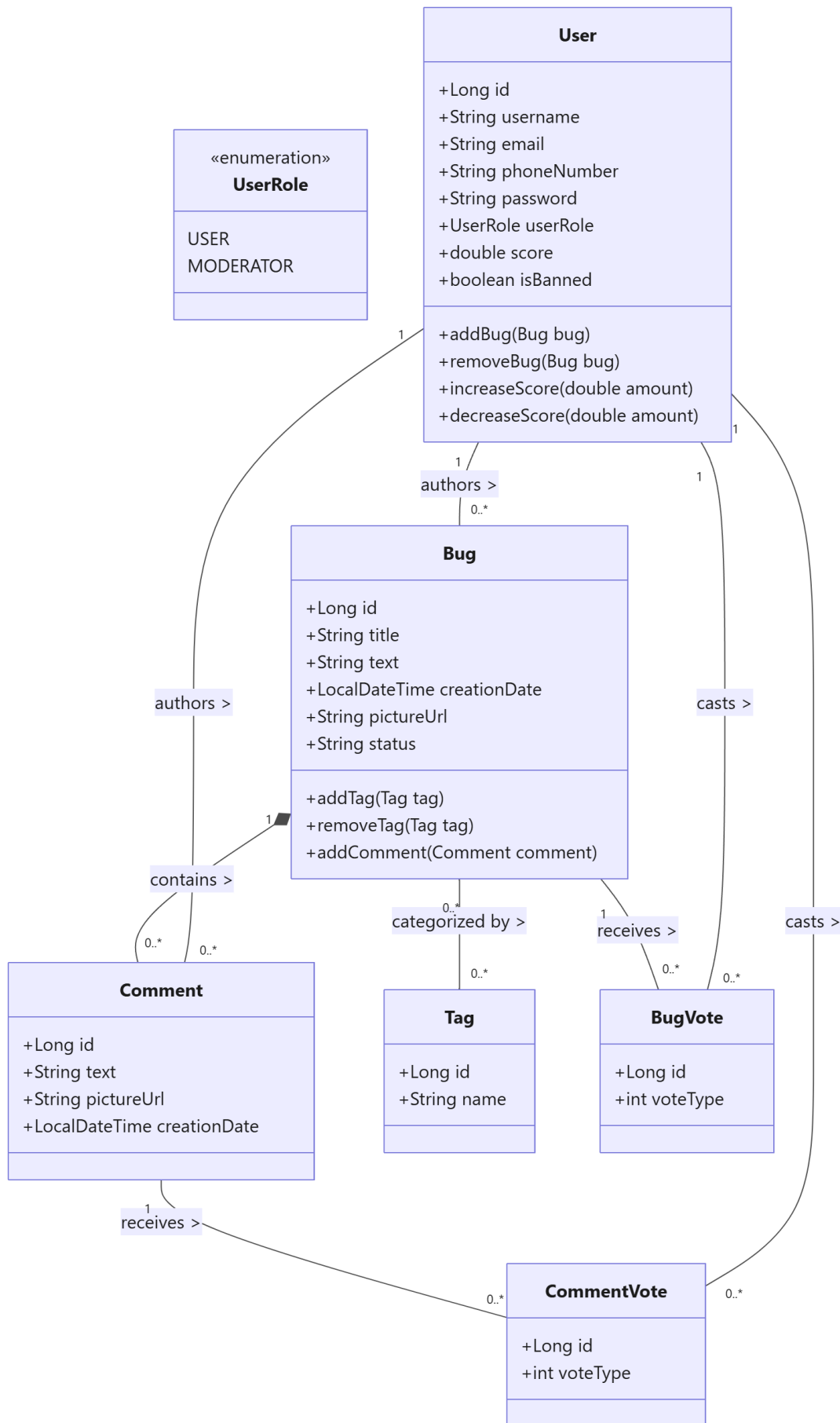


Figure 4: Domain Model Class Diagram showing Entity relationships

- **Server Side (Microservice):** The `NotificationController` orchestrates `EmailService` (via `JavaMailSender`) and `SmsService` (via `RestTemplate` to `SMSPitt`).

Notification Payload

```
{
  "email": "user@local.dev",
  "phoneNumber": "+40712345678",
  "message": "Your account has been banned."
}
```

3.3 API Endpoints

Authentication

Method	Path	Purpose
POST	/auth/login	Authenticate user credentials, create session

Users

Method	Path	Purpose
POST	/users/register	Register a new user account
GET	/users	Retrieve all registered users
GET	/users/{id}	Retrieve a specific user's profile
GET	/users/{id}/score	Retrieve a user's current reputation score
PUT	/users/{id}	Update a user's profile information
DELETE	/users/{id}	Delete a user account

Bugs

Method	Path	Purpose
POST	/bugs	Submit a new bug report
GET	/bugs	Retrieve all bugs (supports query filters)
GET	/bugs/{id}	Retrieve a specific bug with its comments
PUT	/bugs/{id}	Update a bug report (author or moderator only)
DELETE	/bugs/{id}	Delete a bug report (author or moderator only)

Comments & Votes

Method	Path	Purpose
POST	/bugs/{id}/comments	Add a comment to a bug report
GET	/bugs/{id}/comments	Retrieve all comments for a bug
PUT	/comments/{id}	Update a comment (author only)
DELETE	/comments/{id}	Delete a comment (author only)
POST	/bugs/{id}/votes	Cast or toggle a vote on a bug
POST	/comments/{id}/votes	Cast or toggle a vote on a comment

Moderation & Filters

Method	Path	Purpose
POST	/moderation/users/{id}/ban	Ban user account and trigger notification
POST	/moderation/users/{id}/unban	Unban a previously banned user account
GET	/api/tags	Retrieve all tag names for filters
POST	/notifications (Port 8081)	Dispatch email and SMS notification

4 Frontend Implementation

4.1 Component Architecture

The React frontend follows a **feature-based component architecture** organised into directories such as `api/`, `components/`, `context/`, `pages/`, and `types/`.

Pages (Route-Level Components)

Component	Route	Description
LoginPage	/login	Combined login and registration form
BugListPage	/ and /bugs	Main dashboard displaying all bug reports
BugDetailsPage	/bugs/:id	Detailed bug view with comments and voting
UserProfilePage	/profile	Current user's profile and authored bugs
ModerationPage	/moderation	Moderator-only panel for managing users

4.2 Routing and Navigation

Client-side routing is implemented with **React Router DOM v7**. The `App.tsx` component defines the route tree within a `<BrowserRouter>`:

- **Public routes:** `/login` is accessible without authentication.
- **Protected routes:** `/`, `/bugs`, `/bugs/:id`, and `/profile` are wrapped in `<ProtectedRoute>`.
- **Moderator-only routes:** `/moderation` is wrapped in `<ModeratorRoute>`.

5 Evaluation and Reflection

5.1 Future Improvements

Improvement 1: Asynchronous Messaging with a Message Broker

The current monolith-to-microservice communication uses synchronous REST calls. A more robust architecture would replace this with an **asynchronous message broker** (e.g., RabbitMQ or Apache Kafka). The monolith would publish a `BanNotificationEvent` to a queue, providing guaranteed delivery, retry semantics, and full decoupling.

Improvement 2: Pagination and Server-Side Filtering

The current `GET /bugs` endpoint loads all bug reports into memory. A production-ready improvement would introduce server-side pagination using Spring Data's `Pageable`, database-level filtering via JPQL/Criteria API, and infinite scroll on the frontend.